

Problem 1

1A

We kept preprocessing intentionally simple. We relied on CountVectorizer's default tokenizer to lowercase the text, strip punctuation, and extract tokens, giving us a consistent representation without introducing any custom cleaning steps.

The vocabulary was learned automatically by CountVectorizer during training. The `min_df` and `max_df` thresholds determined which words entered the final vocabulary and therefore controlled the dimensionality of the feature space. We excluded extremely common words using the `max_df` hyperparameter, since terms that appear in nearly every document typically add noise rather than predictive value. Although we initially considered removing rare words, our experiments showed that the model performed better when these terms were retained, so we set `min_df` to a low value.

We did not impose a limit on vocabulary size through `max_features`. As a result, the final vocabulary contained 27,655 words. Although the exact vocabulary size can vary slightly across training folds, this number is representative of what we observed during 10-fold cross-validation.

We used word counts instead of binary indicators. Our best-performing model used `binary = False`, meaning that the classifier benefited from frequency information rather than simple presence or absence of words.

Out-of-vocabulary words in the test set are handled by ignoring them. Because the feature vector only contains entries for terms in the learned vocabulary, any unseen word contributes a zero and therefore has no effect on the representation.

Finally, we used off-the-shelf tools from scikit-learn, including CountVectorizer, Pipeline, and GridSearchCV, rather than implementing the Bag-of-Words representation from scratch. This allowed us to focus on hyperparameter tuning and evaluation while relying on well-tested library components for tokenization and feature construction.

1B

We optimized hyperparameters using AUROC as the scoring metric. AUROC is appropriate because it evaluates ranking quality rather than relying on a fixed decision threshold, and it is less sensitive to class imbalance.

We used 10-fold stratified cross-validation. With 5,557 total training examples, each fold contained approximately 556 examples (10%) for validation and the remaining 5,001 examples (90%) for training. Stratification ensured that the proportions of “easy” and “hard” labels remained consistent across all folds.

After cross-validation identified the best hyperparameter configuration, we retrained a final model on the entire training set using those selected hyperparameters. This final model was then applied to the test set to generate predictions.

Again, we relied on scikit-learn tools such as CountVectorizer, Pipeline, StratifiedKFold, and GridSearchCV.

1C

We performed a grid search over hyperparameters controlling both the Bag-of-Words representation and the logistic regression classifier. Logistic regression is well suited for text classification because it performs effectively in high-dimensional sparse feature spaces such as those produced by the BoW representation. With L2 regularization, the model can control overfitting even when the vocabulary contains thousands of features.

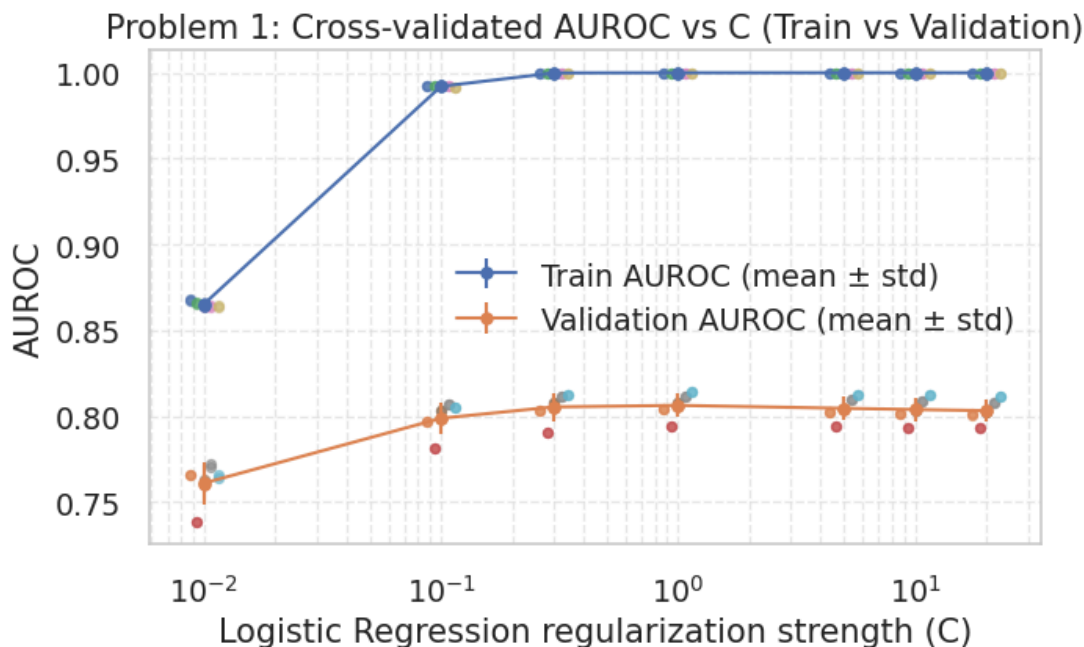
For CountVectorizer, we tuned min_df, max_df, max_features, and binary to study how frequency thresholds shape the vocabulary, how vocabulary size limits affect representation, and whether raw counts or binary indicators provide better features.

For logistic regression, we varied the regularization strength (C) and class weighting (None vs. "balanced"), while using the liblinear solver with an L2 penalty.

The grid values we explored were:

- min_df: 1, 2, 4, 7, 10
- max_df: 0.95, 0.90, 0.975
- max_features: None, 100, 1000, 5000
- binary: True, False
- C: 0.1, 1, 10, 100, 1000
- class_weight: None, "balanced"
- solver: liblinear
- penalty: L2
- cross-validation folds: k = 3, 5, 7, 10

The liblinear solver was chosen because it performs well on relatively small datasets and converges reliably for L2-regularized logistic regression. In our experiments, the solver converged without issues, so no additional step-size tuning or early stopping was required.



The figure above shows that with C-values less than 0.1, the model underfits slightly as demonstrated by the lower training and validation AUROC. Furthermore, the figure illustrates that as the regularization strength decreases with a larger C, training AUROC almost always hovers around 1.0. This indicates that our logistic regression model fits the training set extremely well. However, for the validation set, as C increases, the AUROC plateaus at around 0.8. This indicates that a larger C value does not produce better generalization for our model.

1D

Our model achieved an AUROC of 0.8925 on the test set, which is noticeably higher than the cross-validated AUROC of 0.8139. Typically, validation performance slightly overestimates or closely matches test performance, but in this case, the test set appears to represent an easier generalization task.

One possible explanation is that the training data contains many borderline or mixed-difficulty passages, while the test set includes more clearly separable examples whose vocabulary aligns well with patterns learned during training. This distribution difference could make the test set easier to classify, leading to higher observed performance.

Problem 2:

2A:

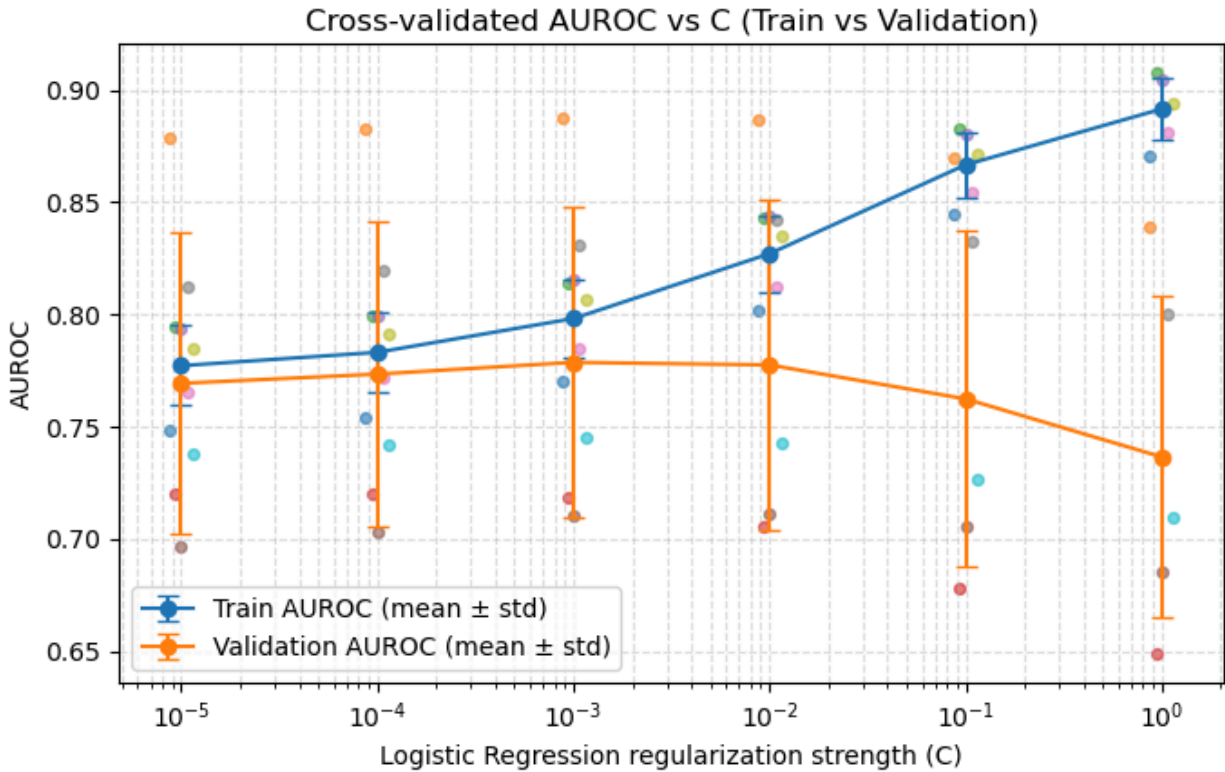
For Problem 2, we built a hybrid feature representation that combines pretrained BERT embeddings with a subset of custom numeric features from `x_train.csv`. We first loaded the 768-dimensional BERT embeddings from `x_train_BERT_embeddings.npz`, where each row corresponds to a passage. Then, to identify the most informative custom features, we trained an L1-regularized logistic regression model on the scaled custom feature matrix using a `StandardScaler` from `sklearn`, and retained only those features whose coefficients remained non-zero. The final feature vector for each passage was formed by combining its BERT embedding with the scaled values of these selected custom features, producing a fixed-length hybrid representation suitable for classification.

2B:

Similar to Problem 1, we used cross-validation on the training set for both hyperparameter selection and performance estimation. After constructing the hybrid feature matrix for all training examples, we applied `GridSearchCV` with 5-fold cross-validation and used AUROC as the scoring metric. The model was trained on a subset of the training data and evaluated on the held-out fold, ensuring that all hyperparameter decisions were based solely on `x_train.csv` and `y_train.csv`. This prevents any use of the test set during model development and provides an unbiased estimate of generalization performance. The final reported “training AUROC” for Problem 2 is the mean cross-validated AUROC returned by `GridSearchCV`.

2C:

For the final classifier, we used logistic regression on the hybrid feature vectors because it is efficient in high-dimensional settings (BERT + custom features), produces well-calibrated probability outputs needed for AUROC, and remains interpretable compared to more complex models. We performed hyperparameter search over the inverse regularization strength C using `GridSearchCV` with 5-fold cross-validation. The search grid included $C = 0.001, 0.01, 0.1, 1, 3, 10$, with `solver = 'lbfgs'` and `max_iter = 5000` to ensure convergence. For each value of C , the model was trained on the hybrid training features and evaluated using AUROC on the held-out fold. We selected the classifier with the highest mean cross-validated AUROC as our final model.



The above figure shows the average training and validation AUROC at each value of C across 5 cross-validation folds. The bars demonstrate the spread of one standard deviation, and the points represent individual AUROC in that fold. The figure shows that past a C-value of 0.01, the training and validation AUROC begins to diverge further. This indicates that for C-values greater than 0.01, the model shows signs of overfitting. Additionally, at C-values below 0.001, the training and validation AUROC remain extremely similar, showing signs of underfitting. The highest validation AUROC occurs at $C = 0.001$, which tells us that stronger regularization provides better generalization in this case.

2D:

We found the Test AUROC reported on Gradescope to be 0.66508 for Problem 1 and 0.72659 for Problem 2. To confidently analyze correct and incorrect predictions on held-out data, we used `sklearn.cross_val_predict` with `method = 'predict_proba'` on the training set for each problem. This produces out-of-fold probability predictions for every training instance, ensuring that each prediction is made when that instance is in a held-out fold. We then ranked examples by predicted probability confidence: for confidently correct cases, we selected examples where the predicted class matched the true label, and the predicted probability was very close to 0 or 1; for confidently incorrect cases, we selected examples where the predicted class was wrong, but the model's probability was still very high.

Most Confident Correct Predictions (held-out via CV)

	author	title \
1360	Emperor of Rome Marcus Aurelius	Meditations
1757	François Rabelais	Gargantua and Pantagruel
4929	Thomas Hobbes	Leviathan

	probability_pos	true_label	pred_label	confidence
1360	0.958946	Key Stage 4-5	Key Stage 4-5	0.458946
1757	0.955082	Key Stage 4-5	Key Stage 4-5	0.455082
4929	0.955000	Key Stage 4-5	Key Stage 4-5	0.455000

Most Confident Incorrect Predictions (held-out via CV)

	author	title
3296	Jules Verne	From the Earth to the Moon; and, Round the Moon
1638	Frank Norris	McTeague: A Story of San Francisco
3246	Jr. Horatio Alger	Phil, the Fiddler

	probability_pos	true_label	pred_label	confidence
3296	0.922861	Key Stage 2-3	Key Stage 4-5	0.422861
1638	0.098394	Key Stage 4-5	Key Stage 2-3	0.401606
3246	0.895357	Key Stage 2-3	Key Stage 4-5	0.395357

For 3296: char count = 500, word count = 101, sentence count = 6,

Problem 1:

Most Confident Correct Predictions (Problem 1, held-out via CV)

	author	title \
413	Aristotle	The Ethics of Aristotle
2247	Helen Hunt Jackson	Ramona
3269	Jr. Horatio Alger	Ragged Dick, Or, Street Life in New York with ...

	probability	true_label	pred_label	confidence
413	0.999996	1	1	0.499996
2247	0.000017	0	0	0.499983
3269	0.000025	0	0	0.499975

Most Confident Incorrect Predictions (Problem 1, held-out via CV)

	author	title	probability
4758	Stendhal	The Red and the Black: A Chronicle of 1830	0.000090
2179	H. G. Wells	The World Set Free	0.000535

2963 Jean Toomer

Cane 0.999064

	true_label	pred_label	confidence
4758	1	0	0.499910
2179	1	0	0.499465
2963	0	1	0.499064

Problem 1 – Most Confident Correct:

Author	Title	Predicted Possibility
Aristotle	The Ethics of Aristotle	0.9999
Helen Hunt Jackson	Ramona	0.000017
Jr. Horatio Alger	Ragged Dick, Or, Street Life in New York with the Boot-Blacks	0.000025

Problem 1 – Most Confident Incorrect:

Author	Title	Predicted Possibility
Stendhal	The Red and the Black: A Chronicle of 1830	0.00009
H.G. Wells	The World Set Free	0.00054
Jean Toomer	Cane	0.99906

Problem 2 – Most Confident Correct:

Author	Title	Predicted Possibility
Emperor of Rome Marcus Aurelius	Meditations	0.95895
François	Gargantua and Pantagruel	0.95508
Thomas Hobbes	Leviathan	0.955

Problem 2 – Most Confident Incorrect:

Author	Title	Predicted Possibility
--------	-------	-----------------------

Jules Verne	From the Earth to the Moon; and, Round the Moon	0.92286
Frank Norris	McTeague: A Story of San Francisco	0.09839
Jr. Horatio Alger	Phil, the Fiddler	0.89536

Examining the most confident correct predictions for Problem 1, its most confident predictions were extremely confident, with the predicted probability being almost exactly 1 or exactly 0. Additionally, its most confident incorrect predictions follow the same pattern, with the predicted probability being almost exactly 1 or exactly 0. These extremely one-sided predictions tell us that our bag-of-words predictions are identifying vocabulary that distinctly deciphers the reading level. Thus, our model from Problem 1 seems to only use lexical cues to identify passages. On the other hand, our model from Problem 2 incorporated more elements into its prediction. All the works that our Problem 2 model most confidently predicted, whether correct or incorrect, were based on abstract vocabulary, longer or shorter sentences, or more complex or basic sentence structures.

It also seemed like the model from Problem 2 tended to perform better on passages where subtle semantic cues and stylistic features both mattered, while the earlier model struggled more with more complex cases. On the other hand, our model for Problem 1 is better at capturing the lexical attributes of a reading level.